Object-Oriented and Classical Software Engineering

8th Edition, WCB/McGraw-Hill, 2010

Stephen R. Schach
srs@vuse.vanderbilt.edu

CHAPTER 4

TEAMS

Overview

- Team organization
- Democratic team approach
- Classical chief programmer team approach
- Beyond chief programmer and democratic teams
- Synchronize-and-stabilize teams
- Teams for agile processes
- Open-source programming teams
- People capability maturity model
- Choosing an appropriate team organization

Team Organization

- A product must be completed within 3 months, but 1 person-year of programming is still needed
- Solution:
  - If one programmer can code the product in 1 year, four programmers can do it in 3 months
- Nonsense!
  - Four programmers will probably take nearly a year
  - The quality of the product is usually lower

Task Sharing

- If one farm hand can pick a strawberry field in 10 days, ten farm hands can pick the same strawberry field in 1 day

- One elephant can produce a calf in 22 months, but 22 elephants cannot possibly produce that calf in 1 month

Task Sharing (contd)

- Unlike elephant production, it is possible to share coding tasks between members of a team
- Unlike strawberry picking, team members must interact in a meaningful and effective way

Programming Team Organization

- Example:
  - Sheila and Harry code two modules, `m1` and `m2`, say
- What can go wrong
  - Both Sheila and Harry may code `m1`, and ignore `m2`
  - Sheila may code `m1`, Harry may code `m2`. When `m1` calls `m2` it passes 4 parameters; but `m2` requires 5 parameters
  - Or, the order of parameters in `m1` and `m2` may be different
  - Or, the order may be same, but the data types may be slightly different

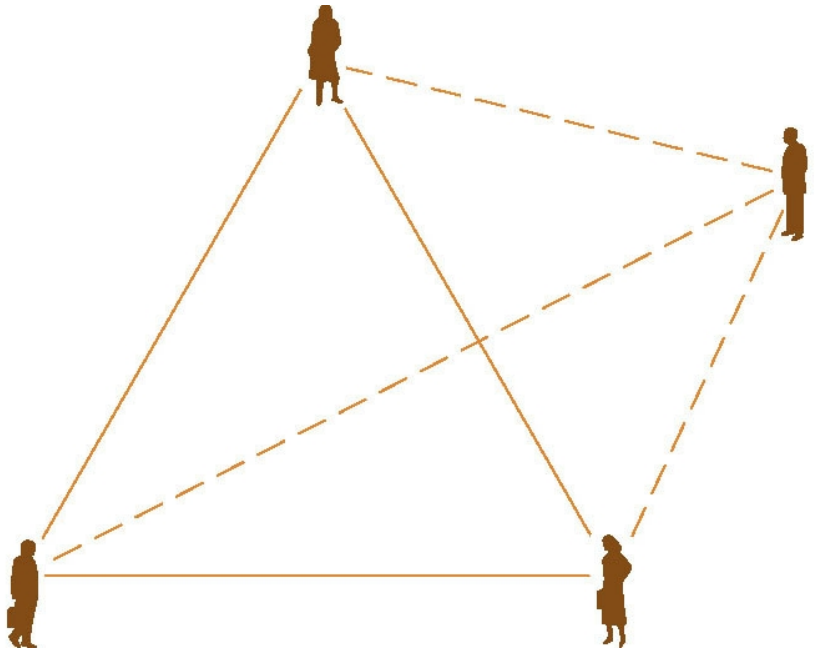
Programming Team Organization (contd)

- This has nothing whatsoever to do with technical competency
  - Team organization is a managerial issue

Communications Problems

- Example
  - There are three channels of communication between the three programmers working on a project. The deadline is rapidly approaching but the code is not nearly complete

- "Obvious" solution:
  - Add a fourth programmer to the team



Communications Problems (contd)

- But other three have to explain in detail
  - What has been accomplished
  - What is still incomplete
- Brooks's Law
  - Adding additional programming personnel to a team when a product is late has the effect of making the product even later

Team Organization

- Teams are used throughout the software production process
  - But especially during implementation
  - Here, the discussion is presented within the context of programming teams
- Two extreme approaches to team organization
  - Democratic teams (Weinberg, 1971)
  - Chief programmer teams (Brooks, 1971; Baker, 1972)

Democratic Team Approach

- Basic underlying concept — *egoless programming*
- Programmers can be highly attached to their code
  - They even name their modules after themselves
  - They see their modules as extension of themselves

Democratic Team Approach (contd)

- If a programmer sees a module as an extension of his/her ego, he/she is not going to try to find all the errors in “his”/“her” code
  - If there is an error, it is termed a *bug* 
  - The fault could have been prevented if the code had been better guarded against the “bug”
  - “Shoo-Bug” aerosol spray

Democratic Team Approach (contd)

- Proposed solution

- Egoless programming
  - Restructure the social environment
  - Restructure programmers' values
  - Encourage team members to find faults in code
  - A fault must be considered a normal and accepted event
  - The team as whole will develop an ethos, a group identity
  - Modules will "belong" to the team as whole
  - A group of up to 10 egoless programmers constitutes a *democratic team*

Difficulties with Democratic Team Approach

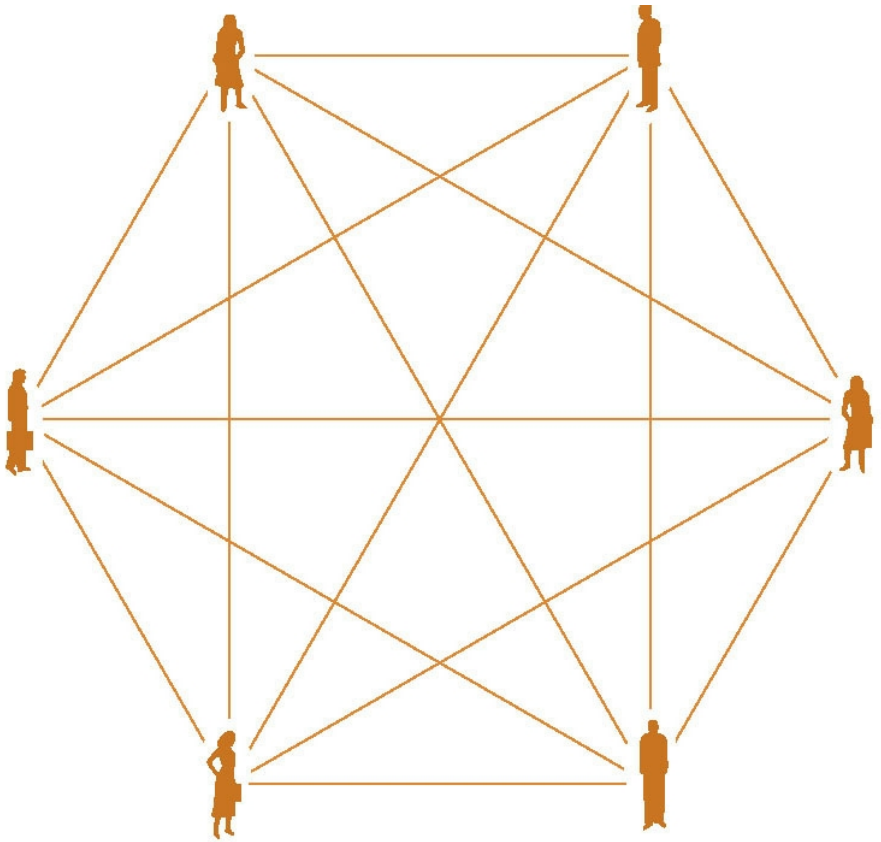
- Management may have difficulties
  - Democratic teams are hard to introduce into an undemocratic environment

Strengths of Democratic Team Approach

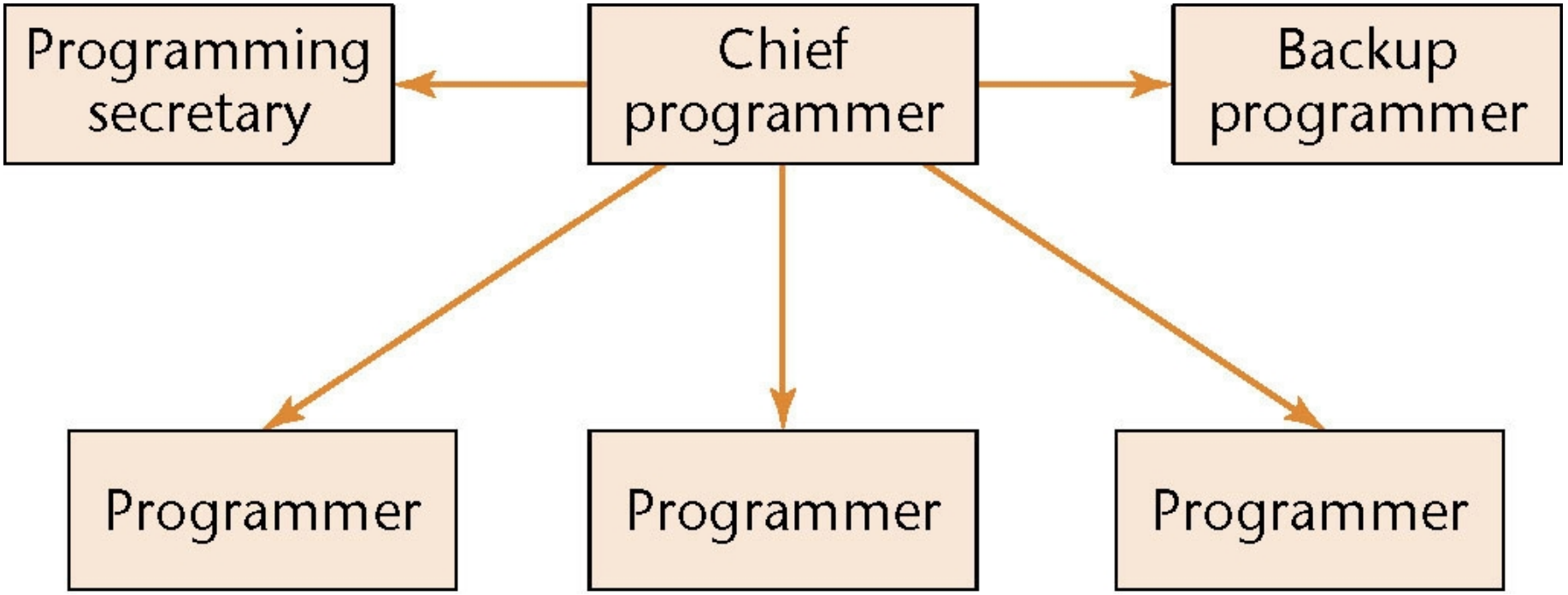
- Democratic teams are enormously productive
- They work best when the problem is difficult
- They function well in a research environment
- Problem:
  - Democratic teams have to spring up spontaneously

Classical Chief Programmer Team Approach

- Consider a 6-person team
  - Fifteen 2-person communication channels
  - The total number of 2-, 3-, 4-, 5-, and 6-person groups is 57
  - This team cannot do 6 person-months of work in 1 month



Classical Chief Programmer Team



- Six programmers, but now only 5 lines of communication

Classical Chief Programmer Team (contd)

- The basic idea behind the concept
  - Analogy: chief surgeon directing an operation, assisted by
    - Other surgeons
    - Anesthesiologists
    - Nurses
    - Other experts, such as cardiologists, nephrologists
- Two key aspects
  - Specialization
  - Hierarchy

Classical Chief Programmer Team (contd)

- Chief programmer
  - Successful manager *and* highly skilled programmer
  - Does the architectural design
  - Allocates coding among the team members
  - Writes the critical (or complex) sections of the code
  - Handles all the interfacing issues
  - Reviews the work of the other team members
  - Is personally responsible for every line of code

Classical Chief Programmer Team (contd)

- Back-up programmer
  - Necessary only because the chief programmer is human
  - The back-up programmer must be in every way as competent as the chief programmer, and
  - Must know as much about the project as the chief programmer
  - The back-up programmer does black-box test case planning and other tasks that are independent of the design process

Classical Chief Programmer Team (contd)

- Programming secretary
  - A highly skilled, well paid, central member of the chief programmer team
  - Responsible for maintaining the program production library (documentation of the project), including:
    - Source code listings
    - JCL
    - Test data
  - Programmers hand their source code to the secretary who is responsible for
    - Conversion to machine-readable form
    - Compilation, linking, loading, execution, and running test cases (this was 1971, remember!)

Classical Chief Programmer Team (contd)

- Programmers
  - Do nothing but program
  - All other aspects are handled by the programming secretary

Impracticality of Classical CPT

- The chief programmer must be a highly skilled programmer *and* a successful manager
- There is a shortage of highly skilled programmers
- There is a shortage of successful managers
- The qualities needed to be a highly skilled programmer are unlikely to be found in a successful manager, and vice versa

Impracticality of Classical CPT **(contd)**

- The *back-up programmer* must be as good as the chief programmer
  - But he/she must take a back seat (and a lower salary) waiting for something to happen to the chief programmer
  - Top programmers, top managers will not do that
- The *programming secretary* does nothing but paperwork all day
  - Software professionals hate paperwork
- Classical CPT is impractical

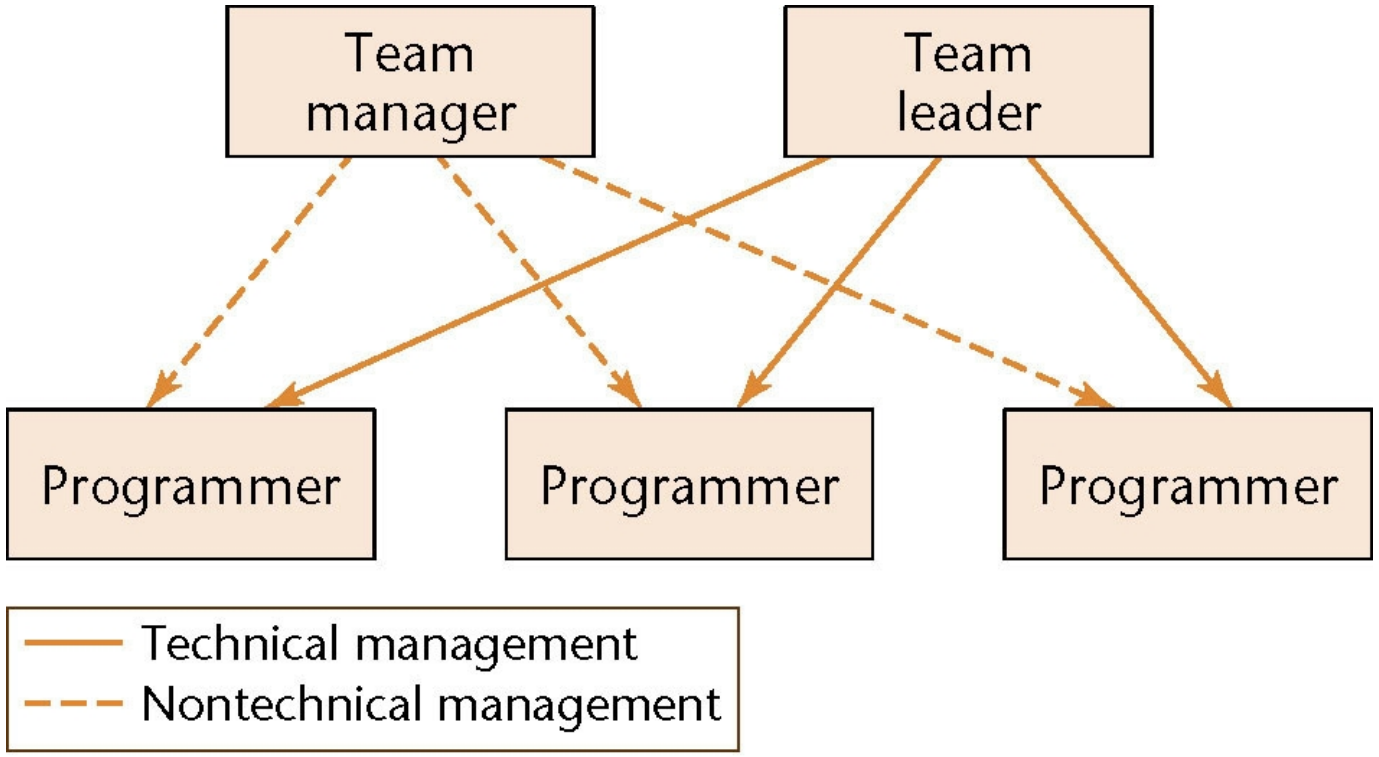
Beyond CP and Democratic Teams

- We need ways to organize teams that
  - Make use of the strengths of democratic teams and chief programmer teams, and
  - Can handle teams of 20 (or 120) programmers
- A strength of democratic teams
  - A positive attitude to finding faults
- Use CPT in conjunction with code walkthroughs or inspections

Beyond CP and Democratic Teams (contd)

- Potential pitfall
- The chief programmer is personally responsible for every line of code
  - He/she must therefore be present at reviews
- The chief programmer is also the team manager
  - He/she must therefore *not* be present at reviews!

Beyond CP and Democratic Teams (contd)



- Solution
  - Reduce the managerial role of the chief programmer

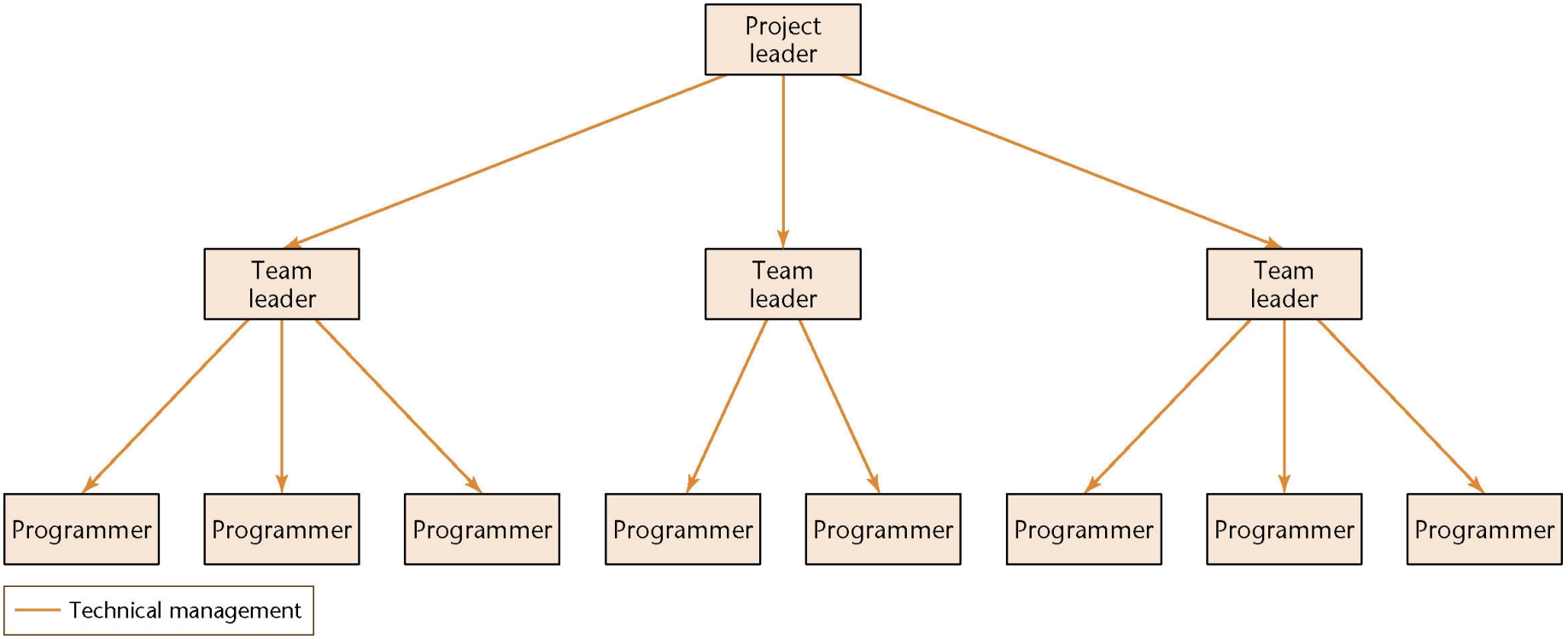
Beyond CP and Democratic Teams (contd)

- It is easier to find a team leader than a chief programmer
- Each employee is responsible to exactly one manager — lines of responsibility are clearly delineated
- The team leader is responsible for only technical management

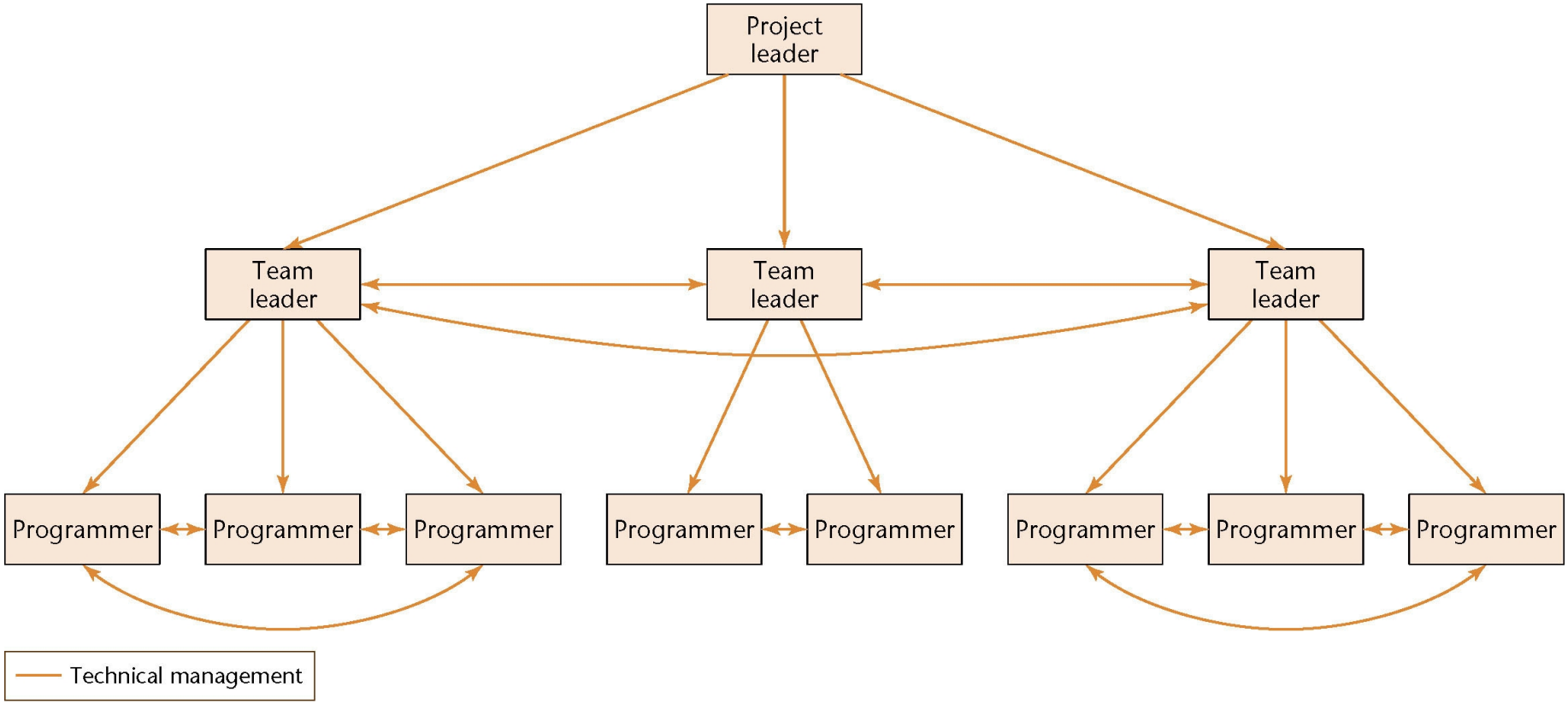
Beyond CP and Democratic Teams (contd)

- Budgetary and legal issues, and performance appraisal are not handled by the team leader
- The team leader participates in reviews — the team manager is not permitted to do so
- The team manager participates in regular team meetings to appraise the technical skills of the team members

Larger Projects



Beyond CP and Democratic Teams (contd)



People Capability Maturity Model

- Best practices for managing and developing the workforce of an organization

- Each maturity level has its own KPAs
  - Level 2: Staffing, communication and coordination, training and development, work environment, performance management, coordination
  - Level 5: Continuous capability improvement, organizational performance alignment, continuous workforce innovation

People Capability Maturity Model (contd)

- P–CMM is a framework for improving an organization's processes for managing and developing its workforce
- No one specific approach to team organization is put forward

Choosing an Appropriate Team Organization

- There is no one solution to the problem of team organization
- The “correct” way depends on
  - The product
  - The outlook of the leaders of the organization
  - Previous experience with various team structures

Choosing an Appropriate Team Organization (contd)

- Exceedingly little research has been done on software team organization
  - Instead, team organization has been based on research on group dynamics in general
- Without *relevant* experimental results, it is hard to determine optimal team organization for a specific product